System Technical Documentation

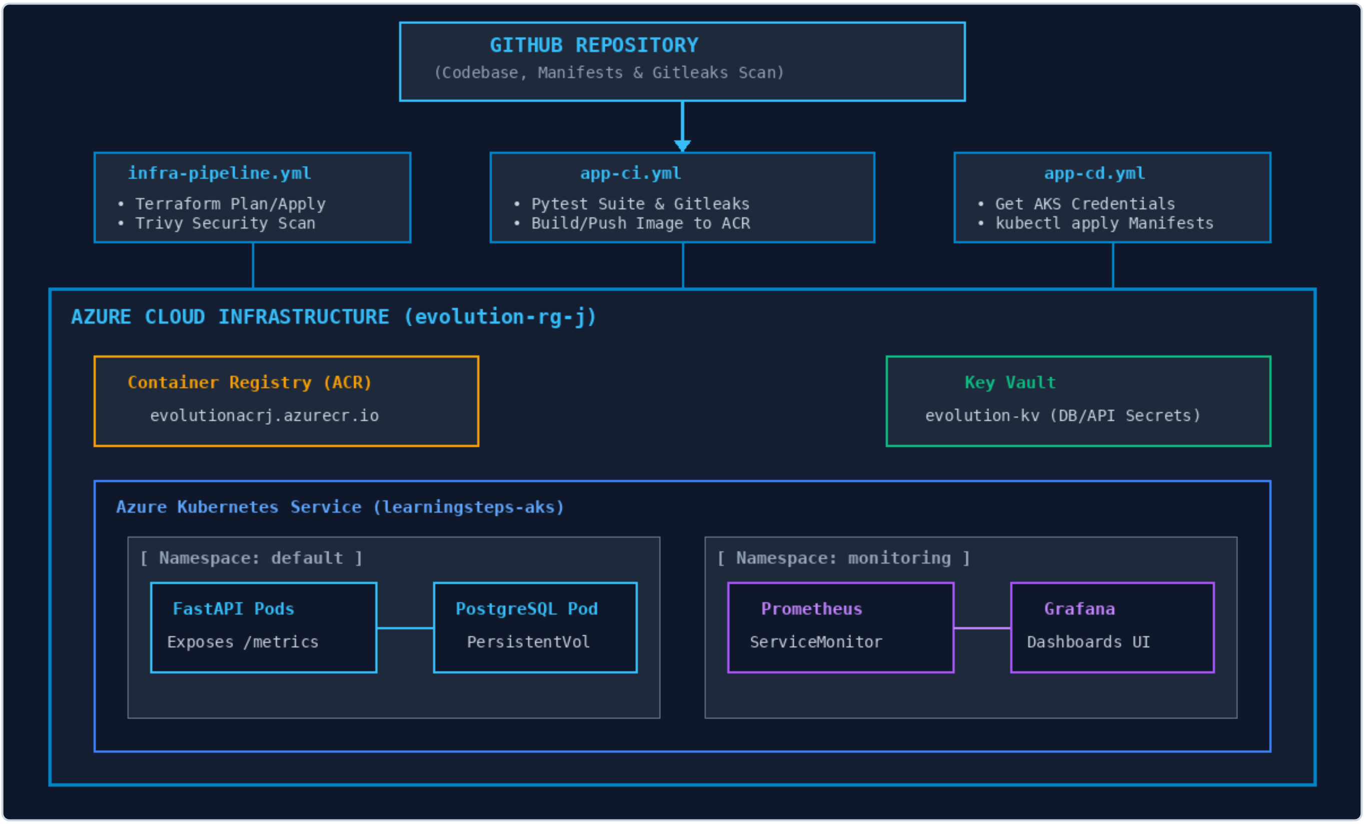
ODINSLAYER09/learningsteps — Azure AKS & DevOps Infrastructure

Repository: github.com/ODINSLAYER09/learningsteps

Date: July 2026 | Version: 2.0.0

1. Executive Overview & System Architecture

The **learningsteps** project is a cloud-native platform deployed on **Microsoft Azure** using **FastAPI (Python)**, **PostgreSQL**, **HashiCorp Terraform**, **Azure Kubernetes Service (AKS)**, **Azure Container Registry (ACR)**, **Azure Key Vault**, **Prometheus**, and **Grafana**.



2. Technical Features & Capabilities

Feature Category	Technical Specification & Description
Application Layer	FastAPI framework, Uvicorn ASGI runner, REST endpoints (`/`, `/health`, `/items`, `/metrics`).
Database Tier	PostgreSQL 15 stateful workload, PersistentVolumeClaim binding, SQLAlchemy ORM models.
Infrastructure-as-Code	Modular HashiCorp Terraform with `azurerm` provider, remote state in Azure Blob Storage.
CI/CD Pipelines	GitHub Actions workflow matrix (Infra, CI, CD), Gitleaks & Trivy security scanning.
Observability Stack	`prometheus-fastapi-instrumentator`, Prometheus Operator, ServiceMonitor CRD, Grafana UI.

3. Pipeline Analysis: In-Scope vs. Out-of-Scope

Workflow File	In-Scope Operations	Out-of-Scope Operations	Automated Quality/Test Checks
`infra-pipeline.yml`	Provisions Resource Group, ACR, AKS, Key Vault, subnets, and RBAC role assignments.	App Docker builds, Kubernetes deployment manifests, application secrets injection.	Terraform syntax validation, `trivy` IaC security scans.
`app-ci.yml`	Runs Python unit tests, builds multi-stage Docker image, logs into ACR via `az acr login`, pushes SHA-tagged image.	Infrastructure provisioning, Kubernetes cluster creation, secret creation in Key Vault.	`pytest` execution, Gitleaks commit security analysis.
`app-cd.yml`	Obtains AKS credentials via `az aks get-credentials`, substitutes image tags into manifests, executes `kubectl apply`.	Database schema creation, cloud infrastructure alterations, Docker image building.	Deployment rollout status checks (`kubectl rollout status`).

4. Azure Prerequisites & Setup Guide

1. Initial Azure Bootstrapping (Manual Steps)

Before running any pipeline, the following bootstrapping resources must be created manually in Azure:

1. **Terraform Remote State Storage:** Create Storage Account (e.g., `stlearningstepstfstate`) and blob container `tfstate` inside Resource Group `rg-tfstate-bootstrap`.
2. **Azure Service Principal Creation:** Generate Service Principal with Subscription Contributor rights:

```
az ad sp create-for-rbac --name "sp-learningsteps-github" --role "Contributor" --scopes /subscriptions/
<AZURE_SUBSCRIPTION_ID> --sdk-auth
```

3. **Role Delegation Permissions:** Grant the Service Principal the `User Access Administrator` (or `Role Based Access Control Administrator`) role on Resource Group `evolution-rg-j` so Terraform can assign `AcrPull` role to AKS.

2. GitHub Repository Secrets Matrix

Secret Name	Description	Example Value
`AZURE_CREDENTIALS`	Full JSON output from Azure Service Principal creation command.	`{"clientId": "...", "clientSecret": "..."}`
`AZURE_CLIENT_ID`	Application Client GUID for Azure Service Principal.	`44f357b8-8232-4bba-a5bf-081ba7a41eb1`
`AZURE_CLIENT_SECRET`	Authentication password for Azure Service Principal.	`wX8~...`
`AZURE_SUBSCRIPTION_ID`	Target Azure Subscription GUID.	`6a9a0ca4-d17c-4caf-a4d5-25747d82995c`
`AZURE_TENANT_ID`	Azure Active Directory Tenant GUID.	`c0531484-50fa-4ff2-97df-b1f2d4810603`
`ACR_NAME` / `ACR_SERVER`	Name and FQDN of Azure Container Registry.	`evolutionacrj` / `evolutionacrj.azurecr.io`
`AKS_CLUSTER_NAME`	Managed Azure Kubernetes Service cluster name.	`learningsteps-aks`

3. Key Vault Secrets Configuration

The following secret keys **must** exist inside Azure Key Vault (`evolution-kv`):

- `postgres-admin-password`: PostgreSQL superuser password.
- `postgres-user-password`: Application runtime database password.
- `api-secret-key`: JWT authentication signature key.

5. Database Schema & Test Seed Data

```sql
-- Database Schema Initialization (database/init.sql)
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS items (
    id SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    description TEXT,
    price NUMERIC(10, 2) NOT NULL,
    owner_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Test Seed Data
INSERT INTO users (username, email) VALUES
('alice_dev', 'alice@example.com'),
('bob_ops', 'bob@example.com') ON CONFLICT DO NOTHING;

INSERT INTO items (title, description, price, owner_id) VALUES
('Cloud Handbook', 'Guide to Azure Kubernetes Service', 49.99, 1),
('DevOps Pipeline Script', 'Automated GitHub Actions workflow template', 19.99, 2) ON CONFLICT DO NOTHING;
```

6. Monitoring & Observability Setup

1. Deploying Prometheus & Grafana via Helm

```bash
# monitoring/monitoring-setup.sh
#!/usr/bin/env bash
set -e

echo "Installing Prometheus Community Helm Repository..."
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

echo "Creating monitoring namespace..."
kubectl create namespace monitoring --dry-run=client -o yaml | kubectl apply -f -

echo "Deploying kube-prometheus-stack..."
helm upgrade --install prometheus prometheus-community/kube-prometheus-stack   --namespace monitoring   --set
grafana.adminPassword="AdminSecurePassword123!"   --set
prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false
```

2. ServiceMonitor Definition (`monitoring/servicemonitor.yaml`)

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: fastapi-servicemonitor
  namespace: monitoring
  labels:
    release: prometheus
spec:
  selector:
    matchLabels:
      app: learningsteps-api
  namespaceSelector:
    matchNames:
      - default
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
```

3. PromQL Query Matrix for Grafana

Dashboard Metric Title	PromQL Query Expression	Metric Type
HTTP Request Volume	`sum(rate(http_requests_total[5m])) by (handler, status)`	Rate Counter
P95 API Latency	`histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))`	Histogram Quantile
App Availability	`up{job="learningsteps-api"}`	Binary Health Status

7. Application Access & Endpoint Operations

Component Interface	Access Protocol / Command	Default Credentials / Details
Public Web API	`http://<AKS_LOADBALANCER_IP>/docs`	Swagger Interactive Documentation
Grafana Portal	`kubectl port-forward svc/prometheus-grafana 3000:80 -n monitoring`	User: `admin` | Pass: `AdminSecurePassword123!`
Prometheus UI	`kubectl port-forward svc/prometheus-kube-prometheus-prometheus 9090:9090 -n monitoring`	Raw PromQL Expression Browser